

A Hybrid Fault-Tolerant Workflow Scheduling With Performance Fluctuated Cloud Resources

Qian Ren  and Guangshun Yao 

Abstract—With the increasing complexity of cloud systems, resource performance fluctuation and failure have become two significant factors that affect task execution in the cloud, particularly for workflow tasks with precedence constraints. The former often leads to uncertain task execution time, while the latter can result in task abortion. Although many cloud workflow scheduling algorithms have been proposed for fault tolerance or uncertain task execution time separately, these two issues simultaneously exist in practice and have not yet been thoroughly explored. This article proposes a hybrid fault-tolerant cloud workflow scheduling algorithm designed to study the failures of cloud resources and uncertain task execution time simultaneously. The proposed scheduling process consists of four phases: preprocessing, initial scheduling, online scheduling, and online adjustment. Two fundamental and widely recognized strategies for fault tolerance, replication and resubmission, are integrated into the algorithm. An elastic resource provisioning mechanism is also designed to adjust active resources dynamically. Performance evaluations on both randomly generated and real-world workflows demonstrate that the algorithm effectively schedules deadline-constrained workflows under uncertain task execution time while guaranteeing fault tolerance.

Index Terms—Cloud workflow scheduling, fault tolerance, performance fluctuation, resubmission, replication.

I. INTRODUCTION

MANY intricate issues are often composed of hundreds or even thousands of tasks with interdependent relationships, and can be referred to as workflows [1], [2]. These workflows are frequently time-bound and necessitate massive and distributed computing resources for punctual execution. In recent times, cloud services have been increasingly utilized for these workflows due to the decrease in utilization costs and flexibility of computing power. Particularly,

Infrastructure-as-a-Service clouds offer preconfigured virtual machines (VMs) for leasing, allowing customers to access nearly unlimited VMs on a pay-per-use basis [2]. While the benefits are substantial, the cloud resources have a high probability of resource failure due to various factors [3], [4]. Furthermore, each cloud server is virtualized into a set of VMs, which share the same hardware resources, such as CPU, I/O, storage, and network. The resource sharing scheme can easily lead to resource interference among different VMs on the same server, resulting in fluctuations in VM performance and subsequently uncertain task execution time [1], [5]. The performance of VMs in the Amazon EC2 cloud has been found to exhibit a 24% variation, as reported in [6]. Both VM failures and performance fluctuations have a significant impact on task scheduling, particularly for the cases with deadline constraints. For tasks that are precedence constrained, such as those in a workflow instance, the impact is even more serious. This is because the completion delay of one task caused by either VM failure or performance fluctuation can spread to the tasks on the same VM and further to their successors [4], [7]. Therefore, providing an effective fault-tolerant mechanism is mandatory for deadline-constrained workflows in resource performance fluctuated cloud systems.

In this paper, we consider the fault-tolerant cloud workflow scheduling problem with performance fluctuated VMs. The tasks in the soft deadline constrained workflow are precedence constrained and they can be allocated to VMs. VMs are differently configured (CPU and memory) and elastically provisioned based on the scheduling requirements, but they are unreliable during usage due to performance fluctuation and failure. Deadlines, elastic resource provisioning, performance fluctuation, and VM failure are taken into account for the problem under study. Replication and resubmission, two fundamental operators in most fault-tolerant strategies, are adopted for fault tolerance. Replication involves submitting primary and replicate copies of a task to different process units simultaneously to achieve fault tolerance. Resubmission tries to find another alternative suitable resource to reexecute the task after a fault occurs [4], [7], [8]. The stochasticity of task execution time is adopted to deal with the uncertain task execution time caused by performance fluctuation.

Failure and performance fluctuation of heterogeneous resources, elasticity of resource provisioning, and deadline for workflow instance make the problem difficult, with the main challenges being: (i) There is a higher probability that workflow instances might be finished before their deadlines if all tasks of these instances are assigned to faster VMs. Faster VMs

Received 28 April 2025; revised 1 November 2025; accepted 13 November 2025. Date of publication 21 November 2025; date of current version 5 February 2026. This work was supported in part by the Anhui Provincial Natural Science Foundation under Grant 2408085MF177, in part by the Key Project of Anhui University Science Research under Grant 2023AH051629, Grant 2023AH040221, and Grant KJ2021A1064, in part by the Open Project of State Key Laboratory for Novel Software Technology under Grant KFKT2025B36, in part by the Anhui Provincial Leading Project for Academic Disciplines under Grant DTR2025051, and in part by the Chuzhou University Research Foundation under Grant 2022XJZD15 and Grant 2022XJYB04. (Corresponding author: Guangshun Yao.)

Qian Ren is with the College of Computer and Information Engineering, Chuzhou University, Chuzhou 239000, China (e-mail: renqian99@chzu.edu.cn).

Guangshun Yao is with the College of Computer and Information Engineering, Chuzhou University, Chuzhou 239000, China, and also with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China (e-mail: yaoguangs@126.com).

Digital Object Identifier 10.1109/TSC.2025.3635177

might be overloaded, which leads to more serious performance fluctuation [9] and a higher risk of failure [10] to these VMs. Either performance fluctuation or failure of VMs leads to a delay in the task completion time. The delay can widely propagate to the tasks on the same VM and further to their successors, and bring the effected workflow instances be completed beyond their deadlines. It is hard to satisfy deadline for workflow scheduling with delay propagation caused by either performance fluctuation or failure of heterogeneous VMs. (ii) Though resubmission and replication are two fundamental operators for fault tolerance, they are suited for execution and scheduling stages respectively, and usually conflict with each other. Generally, a higher resubmission indicates better resource utilization with more time consumed whereas more replicates of tasks imply less time consumed with worse resource utilization. It is not easy to select a fault-tolerant strategy (resubmission, replication or both) for each task to enhance resource utilization and to guarantee task execution time simultaneously. (iii) For workflows with different arrival times, the required resources change dynamically. More resources should become active if more workflows arrive. Some resources are converted into the sleep state if fewer resources are required. It is difficult to dynamically balance the number of resources between active and sleep states.

In this paper, the HFWPF (Hybrid Fault-Tolerant Cloud Workflow Scheduling with Performance Fluctuated Resources) is proposed for the considered problem. Both the failures and performance fluctuation of VMs are considered in HFWPF. Replication and resubmission are integrated in HFWPF for fault tolerance with high resource utilization and less time consumption. An elastic resource provisioning mechanism is designed and incorporated into HFWPF to adjust active resources dynamically. The main contributions of this paper are as follows.

- A novel workflow scheduling architecture is designed to mitigate the delay propagation caused by either performance fluctuation or failure of heterogeneous cloud resources.
- A hybrid fault-tolerant strategy integrating resubmission with replication is designed to balance resource utilization and time consumption for scheduling deadline-constrained workflows with performance fluctuated cloud resources.
- An elastic resource partition mechanism is developed. It is incorporated in HFWPF to dynamically adjust active resources for enhancing resource utilization.

The rest of this paper is organized as follows. The related work is discussed in Section II. In Section III, we describe the cloud systems, workflow and fault model used in this paper. The proposed HFWPF is presented in Section IV and evaluated in Section V, followed by conclusions and future work in Section VI.

II. RELATED WORK

As this study is conducted for fault-tolerant cloud workflow scheduling with performance fluctuated resources, we only focus on researches designed on cloud workflow scheduling in the context of fault tolerance and performance fluctuated resources in this section.

A. Cloud Workflow Scheduling With Fault Tolerance

In the past few decades, many fault-tolerant approaches have been proposed to mitigate the impacts of resource faults. Qiu [11] advised to adopt different fault tolerance strategies for different components in a distributed system based on their statistical rank. Javadi [12] attempted to predict faults based on failure models. Jhawar [13] suggested utilizing third-party fault-tolerant mechanisms. However, these approaches often require specialized knowledge of the target infrastructure.

Resubmission is a widely used strategy for fault tolerance in distributed systems. It submits and executes the interrupted task again on the same or another process unit after a failure. In last decades, lots of resubmission based algorithms have been proposed for workflow scheduling [14], [15]. Less resources are consumed by these resubmission based algorithms. Nonetheless, resubmission can significantly delay the finish time of tasks after failures, which is catastrophic for deadline-constrained workflows. Furthermore, the uncertain task execution time are rarely considered in these algorithms.

Replication is another fundamental and widely recognized technique for fault tolerance and usually consumes less time [7]. It generates many copies (one primary copy and one or more replicate copies) for one task and schedules these copies to different process units. If the process unit with the primary copy is suspended because of a fault, the replicate copies on other process units are executed to guarantee the successful completion of this task [7]. In recent years, many replication based algorithms have been proposed for fault-tolerant cloud workflow scheduling [3], [16], [17]. However, these algorithms typically consume more resources than resubmission based algorithms and have worse resource utilization as additional resources are required for the replicate copies. Moreover, these algorithms are designed based on deterministic resource performance and cannot be used for scheduling with performance fluctuated resources.

Different from the above studies in that it only adopts a single fundamental fault-tolerant strategy, Yao [4], Plankensteiner [8] and Dong [18] applied both replication and resubmission simultaneously for fault-tolerant workflow scheduling. In [4], the deadline of a workflow instance was divided into multiple sub-deadlines for all tasks. Based on the sub-deadline, each task selects one strategy from resubmission and replication to achieve fault tolerance. In [8], the impact of the resubmission of each task was used to adjust the replication size of this task to balance resubmission and replication. In [18], deep reinforcement learning was used to select the appropriate fault tolerant strategy from resubmission and replication for each task. Farid et al. [19] employed replication by running three copies of each task on heterogeneous VMs, and used resubmission by re-executing tasks whose results have low confidence or miss their sub-deadlines. Recently, both Setlur [20] and Li [21] integrated checkpoint-restart with replication and resubmission for fault-tolerant cloud workflow scheduling. Nevertheless, all these algorithms are still based on deterministic resource performance and cannot be used for the problem under this study directly.

B. Cloud Workflow Scheduling With Performance Fluctuated Resources

Recently, both Chen [1], [5] and Liu [2] proposed online scheduling algorithms for deadline-constrained cloud workflows with random arrivals and uncertain task execution time. By employing statistical methods to estimate real task execution times, Cai et al. [22] introduced an online delay-based dynamic algorithm for cloud workflow scheduling with stochastic task execution time. Rodriguez et al. [23] proposed a meta-heuristic method to dynamically rent resource intervals for workflows with stochastic task execution time. To mitigate the effects of performance variation of resources while meeting workflow deadlines, Calheiros et al. [24] utilized budget surplus and idle time of provisioned VMs to replicate workflow tasks. Xing et al. [25] designed a local search-driven periodic scheduling algorithm that uses task waiting time to alleviate the negative effects of uncertain task execution times. Xu et al. [26] and Rajput [27] explored real-time workflow scheduling on multi-cloud systems under stringent deadline constraints and uncertain task execution times. Chen [28] and Lin [29] extended this research to Edge-Cloud environments. Recently, Pan [30] utilized a time-series-based prediction model to predict the time-varying performance of VMs, and then scheduled workflow tasks based on the predicted results. Bader [31] and Yang [32] adopted Bayesian linear regression and multilayer neural networks, respectively, to predict uncertain task execution time. Moreover, several cloud workflow scheduling algorithms have been proposed to optimize multiple objectives in the context of uncertain task execution time. Qin [33] proposed a scheduling algorithm for optimizing workflow execution time and monetary cost. Ye et al. [34] and Taghinezhad-Niar [35] aimed to optimize monetary cost, resource utilization, and workflow success ratio. However, resource faults, which can lead to disastrous results for precedence-constrained tasks in cloud systems, are not considered in these algorithms.

Different from above work, this study considers the fault-tolerant cloud workflow scheduling problem with performance fluctuated resources. Both fault tolerance and performance fluctuation are considered. To the best of our knowledge, the problem considered in this paper has not been studied yet even though it widely occurs in practice.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. System Model

In our model, the cloud systems offer a set of M types of VMs $VM = \{vt_1, vt_2, \dots, vt_M\}$ to customers. For example, Amazon EC2 provides 9 types of computing optimized C6g instances.¹ All VMs of the same type vt_m have the same ideal processing capacity vp_m , while VMs from different types are heterogeneous and have different processing speeds. The n^{th} VM with type vt_m is denoted as v_{mn} . The total number of active VMs with type vt_m is denoted as N_m . All VMs are assumed to be located in the same data center or region, so the

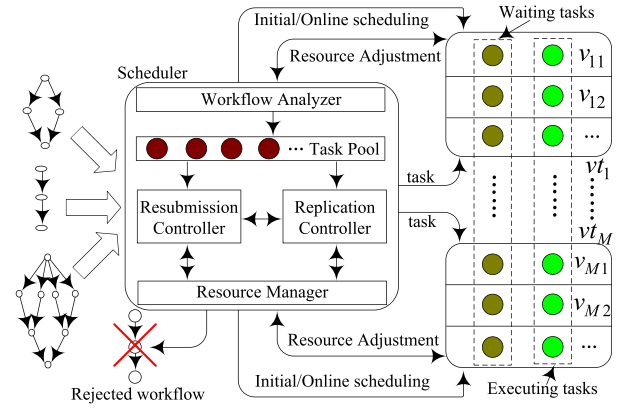


Fig. 1. Scheduling architecture.

network bandwidth NB between VMs is roughly equal [3], [4]. Furthermore, VMs can concurrently receive data when they are executing tasks [1]. Empowered by virtualization technology, cloud systems provide an illusion of unlimited resources [2], [16]. Hence, the number of leased VMs is unlimited in this study. Similar to the elastic resource provision adopted by Amazon EC2, customers can scale the leased VMs according to their requirements. The setup times of VMs are assumed to be identical when they are leased and are denoted by T_{boot} . The VMs are only active during their leased time intervals.

We denote the workflow instances as a set of $W = \{w_1, \dots, w_I\}$. Each workflow instance $w_i \in W$ ($1 \leq i \leq I$) is defined by its arrival time a_i , soft deadline d_i , and structure G_i . The structure of w_i is further modeled as a directed acyclic graph $G_i = (T_i, E_i)$. The set of tasks in w_i is denoted as $T_i = \{t_{is}, t_{i1}, \dots, t_{iJ}, t_{ie}\}$, where t_{is} and t_{ie} are dummy source and sink tasks with 0 execution time. The size of each task t_{ij} ($1 \leq j \leq J$) is denoted as S_{ij} . The set of precedence constraints for tasks in w_i is represented by $E_i = \{(t_{ih}, t_{ij}) | t_{ih} \in T_i, t_{ij} \in T_i, h < j\}$, where $(t_{ih}, t_{ij}) \in E_i$ indicates that task t_{ij} depends on the output of task t_{ih} for its execution. Task t_{ij} can start executing only after receiving all the necessary output data from its predecessors, which ensures the hierarchical parent-child relationships within a workflow instance. The size of transferred data from task t_{ih} to t_{ij} is denoted as D_{hj}^i , and the time used for transferring D_{hj}^i (denoted as TT_{hj}^i) is D_{hj}^i/NB if t_{ih} and t_{ij} are allocated on different VMs, and 0 otherwise. The set of predecessors of task t_{ij} is denoted by $P(t_{ij})$.

For scheduling W , we propose a new scheduling architecture as shown in Fig. 1. It consists of a Workflow Analyzer, a Task Pool (TP), a Resubmission Controller, a Replication Controller, and a Resource Manager. For each arriving w_i , the Scheduler determines whether to accept it through the cooperation of workflow Analyzer and Resource Manager. If the Scheduler cannot find a feasible scheduling strategy for w_i within the given deadline under the ideal state, it rejects w_i . Once w_i is accepted, the proposed HFWPF is invoked. It analyzes the relationship among workflow tasks and then divide the workflow deadline into multiple sub-deadlines for all tasks in w_i . Thereafter, Workflow Analyzer delivers the source task of w_i for scheduling, and

¹<https://aws.amazon.com/ec2/instance-types/>

also stores other tasks into TP. Only one task is allowed to wait on a VM besides the executing task. Therefore, the fluctuation of task execution time on a VM only affects the waiting task on the same VM.

For each ready task t_{ij} in TP, the Scheduler selects a fault-tolerant strategy from resubmission and replication for t_{ij} . Resubmission is selected if t_{ij} has enough time to execute again, otherwise replication is selected. Moreover, the Scheduler also generates an initial allocation result for t_{ij} during the initial scheduling process. If t_{ij} selects replication for fault tolerance, a replicate copy is generated besides the primary one and they are allocated to two different VMs. If t_{ij} selects resubmission, t_{ij} is only allocated to one VM during the initial scheduling process. If t_{ij} is suspended caused by resource failure during the task execution process, the Scheduler adopts an online scheduling scheme to select a suitable resource for t_{ij} . Furthermore, an online adjustment scheme is developed to adjust the available time slots of some unexecuted tasks according to the task execution results during task execution process. It also mitigates the propagated impacts of the uncertain task execution time caused by the performance fluctuation of VMs. Moreover, an elastic resource provisioning mechanism is adopted during the whole process to dynamically adjust active resources. When one active VM containing tasks with the resubmission strategy fails, resubmission is performed immediately.

Differ from deterministic scheduling that assumes the resource performance and task execution time are deterministic, the uncertain scheduling considers the uncertainties in these parameters [1]. Previous work have proven that the task execution time is subject to normal distributions [2], [22]. In this paper, we assume that the task execution time on a performance fluctuated VM is a stochastic variable with independent and normal distributions. We also assume that the expected value and variance of task execution can be estimated by several methods [2], [22], which is beyond the scope of this paper. The approximate weight used in [2] is adopted as the estimation of the task execution time with the stochastic characteristic. Given X as the task execution time, the approximate weight $w(X)$ is defined as

$$w(X) = \begin{cases} E(X) + \sqrt{D(X)} & \text{if } \frac{D(X)}{E(X)^2} \leq 1 \\ E(X) \cdot \left(1 + \frac{1}{\sqrt{D(X)}}\right) & \text{otherwise} \end{cases} \quad (1)$$

where $E(X)$ and $D(X)$ denote the expected value and the variance of variable X respectively. Let $N(\mu, \sigma^2)$ denote the normal distribution with expected value μ and variance σ^2 . As each value of X , $X \sim N(\mu, \sigma^2)$, satisfies $\sigma/\mu^2 \leq 1$, the weight of X is expressed as $w(X) = E(X) + \sqrt{D(X)}$ [2].

Tasks fail when the resource units where they are located fail. In this paper, we focus on the failure of VMs. When a VM fails, all the tasks on this VM cannot be finished. It is also supposed that there exists a fault-detection mechanism, such as heartbeats [15], fail-signal and acceptance test [3], [4], [16], to detect failures. The fault detection time is ignored in this paper. The faults might be transient or permanent and they are assumed to be independent. Each fault only affects a single VM.

Only one VM is assumed to fail simultaneously at any time. The proposed model can be easily extended to the case with multiple VM failures in large-scale cloud environments by dividing the cloud systems into several small groups. In each small group, the proposed model is applied as in [3], [4], [5], [7], [16].

B. Problem Formulation

HFPPF aims to maximize the completion quality of deadline-constrained workflows in performance fluctuated cloud resources with fault tolerance. To formulate the scheduling problem, we first define some notations.

Let boolean variable $z_{ij} \in \{0, 1\}$ represent the fault tolerance strategy adopted by task t_{ij} . $z_{ij} = 1$ means that t_{ij} adopts resubmission and $z_{ij} = 0$ implies that t_{ij} uses replication. There is only one replicate for t_{ij} when $z_{ij} = 0$ and the primary copy t_{ij}^P and the replicate copy t_{ij}^R are executed on two different VMs simultaneously. When $z_{ij} = 0$, v_{ij}^P and v_{ij}^R denote the VMs where t_{ij}^P and t_{ij}^R are allocated respectively. For fault tolerance, $v_{ij}^P \neq v_{ij}^R$. Let boolean variables $x_{mn}^{ij} \in \{0, 1\}$ represent allocations of tasks to VMs. $x_{mn}^{ij} = 1$ if the j^{th} task in w_i is allocated to v_{mn} , which is the n^{th} VM with type vt_m .

Different from deterministic scheduling, the task execution time under this study is uncertain. Only when task t_{ij} is executed by a VM successfully, its actual start time $ast(t_{ij})$ and the actual finish time $aft(t_{ij})$ can be obtained. For a given workflow instance w_i , its actual finish time is equal to $aft(t_{ie})$. We define the completion quality of w_i as $Q_i = \max\{0, 1 - \max\{0, \frac{aft(t_{ie}) - d_i}{d_i - a_i}\}\}$, where a_i and d_i denote the arrival time and soft deadline of w_i respectively. If $aft(t_{ie})$ is less than d_i , Q_i is set to 1. Otherwise, Q_i is measured by the tardiness degree between $aft(t_{ie})$ and d_i . The workflow deadline d_i is a soft deadline, meaning that workflow instance w_i can complete after d_i but with a penalized completion quality Q_i . The average customer satisfaction Q for all accepted workflow instances is $\sum_{i=1}^I Q_i / I$. A greater Q implies that more instances are completed by or near to their deadlines. The considered problem is modelled as follows.

$$\max Q = \frac{\sum_{i=1}^I Q_i}{I} \quad (2)$$

s.t.

$$Q_i = \max \left\{ 0, 1 - \max \left\{ 0, \frac{aft(t_{ie}) - d_i}{d_i - a_i} \right\} \right\}, \quad \forall i \in \{1, \dots, I\} \quad (3)$$

$$aft(t_{ih}) + TT_{hj}^i \leq ast(t_{ij}), (t_{ih}, t_{ij}) \in E_i \quad \forall i \in \{1, \dots, I\}, \forall h \in \{1, \dots, J\}, \forall j \in \{1, \dots, J\} \quad (4)$$

$$\sum_{m=1}^M \sum_{n=1}^{N_m} x_{mn}^{ij} = \begin{cases} 2 & \text{if } z_{ij} = 0 \\ 1 & \text{if } z_{ij} = 1 \end{cases}, \quad \forall i \in \{1, \dots, I\}, \forall j \in \{1, \dots, J\} \quad (5)$$

$$\sum_{i=1}^I \sum_{j=1}^J x_{mn}^{ij} \leq 1, \forall m \in \{1, \dots, M\}, \forall n \in \{1, \dots, N_m\} \quad (6)$$

$$x_{mn}^{ij} \in \{0, 1\}, z_{ij} \in \{0, 1\} \quad (7)$$

Exp. (3) describes the completion quality of w_i with a arrival time a_i and a soft deadline d_i . Exp. (4) indicates the data dependencies of tasks in w_i . Exp. (5) denotes one task with resubmission can be performed only on one VM at a time while one task with replication can be performed only on two VMs with the same VM type at a time. Exp. (6) describes each VM can only perform one task at a time. Exp. (7) denotes the binary variables of scheduling for task t_{ij} and the binary variables of the adoption of a fault-tolerant strategy for t_{ij} .

IV. ALGORITHM IMPLEMENTATION

The proposed HFWPF has four main phases: Preprocessing, Initial Scheduling, Online Scheduling and Online Adjustment. It is motivated on receipt of two monitor signals: the new workflow instance arrival signal and the task execution completion signal.

When receiving a new workflow instance w_i with arrival time a_i and soft deadline d_i , the scheduler takes a simulated scheduling by NOSF [2], which is a cloud workflow scheduling algorithm with uncertain task execution time, and then gets the makespan (sm_{s_i}) before real scheduling. If $d_i - a_i \geq sm_{s_i}$, w_i is accepted and the Preprocessing phase is triggered. Otherwise, w_i is returned to user for revising the deadline. For the accepted w_i , the time interval between a_i and d_i is divided into multiple sub-deadlines for all tasks in w_i . Thereafter, $t_{is} \in T_i$ is delivered to the Initial Scheduling phase, and other tasks are stored into TP.

When receiving a task execution completion signal, the scheduler finds the ready tasks in TP. A workflow task is defined as ready if this task has no predecessor, i.e., $P(t_{ij}) = \phi$, or all the immediate predecessor tasks of this task have been mapped to VMs and at least one predecessor task has been completed [1]. For each ready task t_{ij} in TP, the Initial Scheduling determines its fault-tolerant strategy, and also reserves corresponding resource and serve time.

During the task execution process, if a task with resubmission encounters a fault, the Online Scheduling is triggered to select a suitable VM for executing it again. To make full advantage of time slot, an Online Adjustment scheme is designed to adjust sub-deadlines of some unexecuted tasks. The HFWPF is formally described in Algorithm 1.

A. The Preprocessing Phase

It is obviously that if each task in w_i is completed in/near its sub-deadline, w_i is finished within/near its deadline [4]. For an accepted instance w_i , the Preprocessing phase divides the time slot between a_i and d_i to multiple sub-interval for each task in w_i . Lots of deadline division studies have been reported in last decades [2], [4], [36]. However, the approaches in both [4] and [36] are designed under deterministic task execution time while the method in [2] is tailored for uncertain task execution

Algorithm 1: HFWPF().

```

1 begin
2 if (receive a workflow instance  $w_i$  arrival signal)
3   then
4     Schedule  $w_i$  by NOSF and get  $sm_{s_i}$ ;
5     if ( $d_i - a_i \geq sm_{s_i}$ ) then
6       Call PrePro( $w_i, a_i, d_i$ );
7       Call InitialSch( $t_{is}, [st_{is}, dl_{is}]$ );
8       Put other tasks of  $w_i$  into TP;
9     else
10      Prompt user to specify a deadline above
11       $sm_{s_i}$ ;
12 if ((receive a task completion signal) and
13    ( $TP \neq \emptyset$ )) then
14   if (task  $t_{ij}$  is ready) then
15     Call InitialSch( $t_{ij}, [st_{ij}, dl_{ij}]$ );
16      $TP \leftarrow TP - \{t_{ij}\}$ ;
17   task  $t_{ij}$  is executed on VM  $v_{mn}$ ;
18   if (( $v_{mn}$  encounters a fault) and ( $z_{ij} == 1$ )) then
19     Call OnlineSch( $t_{ij}, [st_{ij}, dl_{ij}]$ );
20   Call OnlineAdj( $P_{ik}, [st_{ik}, dl_{ik}]$ );
21 return.
```

time. Therefore, we adopt the method proposed in [2] in this study. More details about the algorithm for deadline division, i.e., PrePro() in Line 5 in Algorithm 1, can refer to [2]. Thereafter, each task t_{ij} in T_i has a initial available time slot $[st_{ij}, dl_{ij}]$, where st_{ij} and dl_{ij} are the start time and sub-deadline of t_{ij} respectively.

According to [2], the time complexity of PrePro() for processing an arrival workflow w_i is $O(J^2)$, where J is the number of tasks in workflow w_i .

B. Initial Scheduling

For each ready task in TP, Initial Scheduling selects a corresponding fault-tolerant strategy from either resubmission or replication for this task. It also schedules this task to suitable VMs. In the following, we introduce some definitions for Initial Scheduling. Let $w(et_{mn}^{ij})$ denote the approximation weight of the stochastic execution time et_{mn}^{ij} for task t_{ij} on VM v_{mn} and can be calculated by Exp. (1). Let $\psi(t_{ij})$ denote the minimum et_{mn}^{ij} for task t_{ij} on all active VMs, i.e., $\psi(t_{ij}) = \min_{m \in \{1, \dots, M\}} \{\min_{n \in \{1, \dots, N_m\}} \{et_{mn}^{ij}\}\}$.

Previous studies have shown that resubmission is a time consuming strategy for fault tolerance [4], [8]. In this paper, a workflow task $t_{ij} \in T_i$ selects resubmission as its fault-tolerant strategy and z_{ij} is set to 1 if

$$dl_{ij} - st_{ij} \geq 2 \times (T_{boot} + \max_{t_{ih} \in P(t_{ij})} \{TT_{hj}^i\} + \psi(t_{ij})), \quad (8)$$

in which T_{boot} is the VM initial boot time. Otherwise, t_{ij} selects replication and z_{ij} is set to 0. When $z_{ij} = 1$, we define the execution before and after fault as initial execution and reexecution respectively.

After selecting a fault-tolerant for t_{ij} , it is also need to schedule t_{ij} to suitable VMs. When $z_{ij} = 0$, two VMs are required for t_{ij}^P and t_{ij}^R respectively for fault tolerance. The sub-deadlines for t_{ij}^P and t_{ij}^R are identical and equal to dl_{ij} . When $z_{ij} = 1$, if a VM fault happens during the initial execution process of t_{ij} , the reexecution process is required to be completed before its sub-deadline dl_{ij} . The minimum time for the reexecution of t_{ij} (denoted as rex_{ij}) should contain T_{boot} , $\max_{t_{ih} \in P(t_{ij})} \{TT_{hj}^i\}$ and $\psi(t_{ij})$ at least, i.e.,

$$rex_{ij} = T_{boot} + \max_{t_{ih} \in P(t_{ij})} \{TT_{hj}^i\} + \psi(t_{ij}). \quad (9)$$

Therefore, the sub-deadline for the initial execution of t_{ij} is $dl_{ij} - rex_{ij}$.

After selecting the fault tolerant strategy and deciding the sub-deadline for ready task t_{ij} , Initial Scheduling needs to find a suitable VM for t_{ij} . It first tries to allocate t_{ij} to an active VM. If no active VM can satisfy the scheduling requirement of t_{ij} , a new VM is activated and t_{ij} is allocated to this VM.

For ready task t_{ij} to be allocated on active VM v_{mn} , we use pst_{mn}^{ij} to denote the predicted start time of task t_{ij} as

$$pst_{mn}^{ij} = \max \left\{ pft_{mn}^{xy}, \max_{t_{ih} \in P(t_{ij})} \{aft_{m'n'}^{ih} + TT_{hj}^i\} \right\}, \quad (10)$$

where pft_{mn}^{xy} and $aft_{m'n'}^{ih}$ indicate the predicted finish time of task t_{xy} on VM v_{mn} and the actual finish time of task t_{ih} on VM $v_{m'n'}$ respectively. t_{ih} is one predecessor of t_{ij} . $TT_{hj}^i = 0$ if $v_{mn} = v_{m'n'}$. The predicted finish time pft_{mn}^{ij} of task t_{ij} on VM v_{mn} can be expressed as

$$pft_{mn}^{ij} = pst_{mn}^{ij} + w(e_{mn}^{ij}). \quad (11)$$

For one ready task t_{ij} with time slot $[st_{ij}, dl_{ij}]$ and one active VM v_{mn} , if

$$pst_{mn}^{ij} \geq st_{ij} \text{ and } dl_{ij} \geq pft_{mn}^{ij} \quad (12)$$

is satisfied, v_{mn} becomes one ready VM for t_{ij} and is added into RV_{ij} , which denotes the set of ready VMs for t_{ij} . If multiple active VMs satisfy Exp. (12), different selections are adopted for tasks with different fault-tolerant strategies. If $z_{ij} = 1$, in order to reserve enough time for the reexecution, the ready VM with maximum processing capacity is selected. If $z_{ij} = 0$, the ready VM with minimum processing capacity is selected to reduce the user cost as the VMs with higher processing capacity are usually more expensive. It is worthy noting that $v_{ij}^P \neq v_{ij}^R$ if $z_{ij} = 0$.

If no active VM can satisfy Exp. (12), some new VMs (the predicted start time of these VMs is T_{boot}) that meet Exp. (12) are activated and t_{ij} is allocated to these VMs. However, if no VM can satisfy Exp. (12) even new VMs are activated, i.e., $dl_{ij} < pft_{mn}^{ij}$, the VM with the closest value to dl_{ij} is selected as exceeding one subdeadline for a specific task does not mean missing of the whole workflow instance deadline.

The Initial Scheduling procedure is formally described in Algorithm 2. The pseudo-code for Allocate() used in Algorithm 2 is described in Algorithm 3. As two different VMs are required for task with replication while only one VM is required for task with resubmission, the scheduling process for task with replication is different from the process for task with resubmission as shown in Algorithm 3.

Algorithm 2: InitialSch().

Input: $t_{ij}; st_{ij}; dl_{ij}$

Output: the Initial Scheduling for each task;

```

1 begin
2   if (Exp.(8) is satisfied) then
3      $z_{ij} \leftarrow 1$ ;  $dl_{ij} \leftarrow dl_{ij} - rex_{ij}$ ;
4     Call Allocate( $t_{ij}, st_{ij}, dl_{ij}, z_{ij}$ );
5   else
6      $z_q \leftarrow 0$ ;
7     Call Allocate( $t_{ij}, st_{ij}, dl_{ij}, z_{ij}$ );
8   return.

```

Algorithm 3: Allocate().

Input: $t_{ij}; st_{ij}; dl_{ij}; z_{ij}$;

Output: Scheduling result for t_{ij} ;

```

1 begin
2    $RV_{ij} \leftarrow \emptyset$ ;
3   for  $m = 1$  to  $M$  do
4     for  $n = 1$  to  $N_m$  do
5       Calculate  $pst_{mn}^{ij}$  and  $pft_{mn}^{ij}$  by Exp.(10) and
6       Exp.(11) respectively;
7       if (Exp.(12) is satisfied) then
8          $RV_{ij} \leftarrow RV_{ij} \cup \{v_{mn}\}$ ;
9   if ( $z_{ij} == 0$ ) then
10    if ( $|RV_{ij}| \geq 2$ ) then
11      Allocate  $t_{ij}^P$  and  $t_{ij}^R$  in turn to the two VMs
12      with the minimum processing capacity in
13       $RV_{ij}$ ;
14    if ( $|RV_{ij}| == 1$ ) then
15      Allocate  $t_{ij}^P$  to this VM in  $RV_{ij}$ ;
16      Active a new VM that meets Exp.(12) and
17      allocate  $t_{ij}^R$  to this VM;
18    if ( $|RV_{ij}| == 0$ ) then
19      Active two new VMs that meet Exp.(12) and
20      allocate  $t_{ij}^P$  and  $t_{ij}^R$  to these VMs respectively;
21  else
22    if ( $|RV_{ij}| \geq 1$ ) then
23      Allocate  $t_{ij}$  to the VM with maximum
24      processing capacity in  $RV_{ij}$ ;
25    else
26      Active a new VM that meets Exp.(12) and
27      allocate  $t_{ij}$  to this VM;
28  return.

```

In analyzing the time complexity of Algorithm 2 for processing workflow w_i , it becomes evident that the complexity of Lines 6–7 aligns with that of Lines 3–4 within the same algorithm. Consequently, we focus on the time complexity of Lines 2–4 in Algorithm 2. Let N_M represent the number of active VMs with VM type vt_M , which holds the maximum number of active VMs across all types, and let N_{in} be the maximum indegree among the tasks in w_i . For each task t_{ij} , Algorithm 2

necessitates traversing all active VMs to compute $\psi(t_{ij})$, which is subsequently used in Exp. 8 (line 2 in Algorithm 2). This computation requires time $O(JMN_M)$ at the worst-situation, where M denotes the number of VM types in our model. The temporal complexity of line 3 in Algorithm 2 stands at $O(1)$. The complexity of line 4 in Algorithm 2 is determined by Algorithm 3. Within lines 3-7 of Algorithm 3, task t_{ij} needs to traverse its all predecessors and all active VMs for calculating pst_{mn}^{ij} and pft_{mn}^{ij} by Exp. (10) and Exp. (11). These calculated values are then compared with all active VMs by Exp. (12). As a result, under worst-case scenarios, the time complexity for lines 3-7 in Algorithm 3 is $O(N_{in}MN_M)$. Regarding the remainder of Algorithm 3, the time required traversing the set of ready VMs RV can be regarded as a constant since the number of VMs in RV is far less than that of active VMs. In summation, the time complexity of Algorithm 3 is $O(N_{in}MN_M)$, and thus, the complexity of Algorithm 2 coalesces to $O(JMN_M + 1 + N_{in}MN_M) = O((J + N_{in})MN_M)$.

C. Online Scheduling and Online Adjustment

During the task execution process, when $z_{ij} = 0$, t_{ij} is always completed successfully even if one VM failure or performance fluctuation occurs. This is because that two VMs are selected to execute t_{ij}^P and t_{ij}^R simultaneously and only one copy may meet a fault under the proposed model. When $z_{ij} = 1$, t_{ij} is also finished if no fault occurs during its initial execution processes. If a fault happens, it is necessary to find another VM for the first reexecution of t_{ij} as its deadline can allow it to execute at least twice.

Let FT_{ij} denote the time at which the fault is detected during the initial execution processes of t_{ij} with resubmission. Obviously, $dl_{ij} > FT_{ij} > st_{ij}$. Afterwards, we set the start time and sub-deadline of t_{ij} as FT_{ij} and dl_{ij} for the first reexecution of t_{ij} respectively. Algorithm 3 is used again to find a suitable VM for the first reexecution of t_{ij} .

If no fault occurs during the first reexecution of t_{ij} , t_{ij} is completed successfully. However, if the first reexecution of t_{ij} encounters another fault at time FT'_{ij} , the remaining time for t_{ij} is $dl_{ij} - FT'_{ij}$. In this case, it is required to reselect the fault-tolerant strategy for t_{ij} as its remaining available time may not be enough to get another reexecution done. The start time and sub-deadline of t_{ij} are set as FT'_{ij} and dl_{ij} respectively. Then, the same procedure in Algorithm 2 is used to select a suitable fault-tolerant strategy and find a suitable VM for t_{ij} . If a resource failure occurs during the second and subsequent re-executions of t_{ij} , the same process described above will still be followed.

The Online Scheduling procedure is formally described in Algorithm 4. Noticeably, both the Initial Scheduling and Online Scheduling incorporate elastic resource provisioning to improve resource utilization as shown in Algorithms 3 and 4. Similar to previous works such as [2] and [4], new VMs are activated if the active VMs cannot satisfy the scheduling requirements. However, unlike these works kept VMs active during their leased time intervals, in the HFWPF algorithm, VMs become active only when they execute tasks or are waiting for tasks, and the waiting time is less than T_{boot} . Otherwise, the VMs are shut

Algorithm 4: OnlineSch().

Input: $t_{ij}; st_{ij}; dl_{ij}; z_{ij}$
Output: the Online Scheduling for t_{ij} ;
1 **begin**
2 **if** (it is the first fault for t_{ij}) **then**
3 $st_{ij} \leftarrow FT_{ij}$;
4 Call Allocate($t_{ij}, st_{ij}, dl_{ij}, z_{ij}$);
5 **else**
6 $st_{ij} \leftarrow FT'_{ij}$;
7 Call Algorithm2;
8 **return**.

Algorithm 5: OnlineAdj().

Input: $P_{ik}; st_{ik}; dl_{ij}$
Output: the adjusted st_{ik} for t_{ik} ;
1 **begin**
2 **for** (each $t_{ij} \in P(t_{ik})$) **do**
3 Monitor and get aft_{ij} ;
4 $st_{ik} \leftarrow \max_{t_{ij} \in P(t_{ik})} \{aft(t_{ij}) + TT_{jk}^i\}$;
5 **return**.

down immediately to enhance resource utilization and reduce the likelihood of VM failures caused by prolonged idleness.

After all the predecessor tasks of t_{ik} have been completed, the start time st_{ik} of t_{ik} is adjusted to

$$st_{ik} = \max_{t_{ij} \in P(t_{ik})} \{aft(t_{ij}) + TT_{jk}^i\}. \quad (13)$$

The Online Adjustment procedure is formally described in Algorithm 5. By the above scheduling, the HFWPF tries to complete each workflow task within or near its sub-deadline as it reserves corresponding serve time interval for every possible resource fault and performance fluctuation. In cases where some considered faults and performance fluctuations do not occur in the actual execution process, some tasks can finish before their sub-deadlines. These time intervals between the actual finish times and sub-deadlines are then used to expand the deadlines of waiting workflow tasks through the proposed adjustment scheme, effectively alleviating their time constraints. Furthermore, this scheme promotes unexecuted tasks to switch from replication to resubmission as a means of fault tolerance, resulting in reduced resource consumption.

We now analyze the complexity of Algorithms 4 and 5 respectively. The time complexity of Algorithm 4 is obversely equal to that of Algorithm 2, i.e., $O((J + N_{in})MN_M)$. For Algorithm 5, each task $t_{ij} \in T_i$ needs to traverse its all predecessors for obtaining aft_{ij} in Line 3, which is used to adjust st_{ik} in Line 4. Therefore, the time complexity of Algorithm 5 is $O(JN_{in})$ at the worst-situation.

V. PERFORMANCE EVALUATION

To the best of our knowledge, there have been no studies on fault-tolerant cloud workflow scheduling with performance fluctuating resources. In this paper, we conduct a series of

experiments to demonstrate the performance of HFWPF. We compare it with one of its variants, NEHFWPF (Non-Elastic-HFWPF), which refrains from utilizing elastic resource provisioning and maintains all VMs active for scheduling. The quantity of active VMs is set to match the maximum number of tasks at each layer in the test workflow instance. Additionally, we compare HFWPF with two existing cloud workflow scheduling algorithms: NOSF [2] and ICFWS [4]. NOSF is an online multi-workflow Scheduling Framework for scheduling deadline-constrained workflows with random arrivals and uncertain task execution time. Its primary objective is to minimize rental costs while enhancing resource utilization. The scheduling process in NOSF consists of three phases: workflow preprocessing, VM allocation and feedback process. In the initial phase, the framework determines task priorities for each incoming workflow instance. The second phase involves heuristically assigning a VM with available billing time to tasks that are prepared for scheduling. The final phase adaptively adjusts the originally assigned priorities and subdeadlines of the tasks ready for scheduling, such that the deadline of the workflow instance can be met. Notably, NOSF does not account for cloud resource failures. Consequently, in our experimental scenario where a task in NOSF experiences a resource failure, it is dynamically reassigned to another suitable VM during the experiments. ICFWS is designed for a single workflow under deterministic task execution time and uses a similar fault-tolerance method as the proposed HFWPF. To enable ICFWS to handle multiple workflows, we trigger it to schedule all workflow tasks as soon as a new workflow arrives. The above algorithms are compared from the following metrics:

- 1) Quality of Task Completion (*QTC*) is defined as Exp. (2), reflecting the task completion quality of the compared algorithms in the context of soft deadlines, faults and resource performance fluctuation.
- 2) VMs Reserve Time Rate (*VRTR*) is defined as the ratio of the total active time of all VMs in the context of fault and performance fluctuation over the total serve time of all VMs under ideal status got by HEFT [37], reflecting the system resource consumption cost of the compared algorithms under fault and performance fluctuation.

Similar to [3], [4], [5], [7], [16], the following assumptions are adopted in our experiments: (1) All VMs are co-located within the same data center, (2) All VMs are interconnected with identical and stable network, (3) All VMs exhibit uniform startup times, (4) Only a single VM fails at any given instance, and each VM failure is an independent event. Besides the task execution time, our experiments account for the time required to transfer data between pairs of VMs, denoted as TT_{hj}^i in Exps. (4), (8), (9), and (10), and TT_{jk}^i in Exp. (13). Additionally, our experiments consider the setup times of VMs, represented by T_{boot} in Exps. (8) and (9). However, the duration for cloud fault detection is not included, as effective fault detection algorithms can identify system failures within just one second [38], [39]. The subsequent sections detail experiments conducted under various conditions, each replicated 100 times, with the reported results representing the averaged outcomes.

TABLE I
CONFIGURATIONS FOR THE USED VMs

VM type	vCPU	Processing capacity (MFLOPS)	Memory (GiB)	Storage (GB)
c3.large	2	8800	3.75	2 x 16
c3.xlarge	4	17600	7.5	2 x 40
c3.2xlarge	8	35200	15	2 x 80
c3.4xlarge	16	70400	30	2 x 160
c3.8xlarge	32	140800	60	2 x 320

TABLE II
PARAMETERS FOR THE GENERATED WORKFLOWS

Parameter	Value
<i>task number</i>	6000
<i>width</i>	[0.2, 0.4, 0.8]
<i>regularity</i>	[0.2, 0.4, 0.8]
<i>density</i>	[0.2, 0.4, 0.8]
<i>jumps</i>	[1, 2, 3]

A. Evaluation on Simulated Cloud Platform With Randomly Generated Workflows

1) *Experimental Setting*: To observe the performance of the compared algorithms, we first conduct the experiments on randomly generated workflows, which is published by the DAG generator tool.² Five parameters, i.e., *task number*, *width*, *regularity*, *density* and *jumps*, are used to define the workflow instance shape as shown in Table II. More detail about these parameters can be found in [3], [4]. To clearly analyze the performance of the compared algorithms, the experiments are conducted on WorkflowSim [40], which is a popular simulation toolkit for cloud workflow scheduling [3], [4], [15]. 5 types of VMs are simulated for experiments as shown in Table I. The VM configurations are set by referring to the instances in Amazon EC2.³ According to [7], the initial boot time for a VM (T_{boot}) is set to 15 seconds, and the network bandwidth (NB) between VMs is configured to be 10 Gbps.

For each generated workflow instance, the deadline is set as follows.

$$d_i = a_i + \alpha \times sms_i, \quad (14)$$

where α is a deadline factor which reflects how tight or loose the workflow deadlines are. For task t_{ij} located on VM v_{mn} with ideal processing capacity vp_m , its task execution time et_{mn}^{ij} is modelled as a stochastic variable following the normal distribution with the expected value $\mu(et_{mn}^{ij})$ and the variance $\sigma(et_{mn}^{ij})^2$, we set $\mu(et_{mn}^{ij})$ and $\sigma(et_{mn}^{ij})$ as

$$\mu(et_{mn}^{ij}) = \frac{S_{ij}}{vp_m}, \sigma(et_{mn}^{ij}) = \mu(et_{mn}^{ij}) \times \beta, \quad (15)$$

where β is a variance factor that indicates the fluctuation of the task execution time caused by the performance fluctuation of VMs. In our experiments, the task execution time et_{mn}^{ij} is defined using a normal random number generator with the expected value of $\mu(et_{mn}^{ij})$ and the standard deviation of $\sigma(et_{mn}^{ij})$.

²<https://github.com/frs69wq/daggen>

³<https://aws.amazon.com/cn/ec2/previous-generation/>

TABLE III
PARAMETERS OF THE SIMULATION

Parameter	Value (fixed)-(min, max, step)
α	(1.5)-(0.5, 2.5, 0.5)
β	(0.3)-(0.1, 0.5, 0.1)
$FR(\%)$	(7.5)-(2.5, 12.5, 2.5)
CCR	(1)-(0.8, 1.2, 0.1)

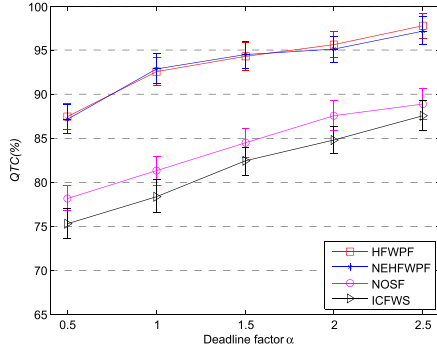


Fig. 2. Performance impact on QTC as a function of α .

For each workflow instance, four parameters are utilized for evaluation: the deadline factor α , the task execution time variance factor β , the VM failure rate (FR), and the communication to computation ratio (CCR). The CCR is defined as the ratio of the average communication cost to the average computation cost. A higher CCR indicates a relatively greater communication load compared to the computation load. The first parameter reflects the impact of the workflow instance deadline on the performance of the algorithms being compared. The subsequent two parameters are used to indicate the influence of resource characteristics, such as performance fluctuations and failures, on the performance. The final parameter assesses the impact of varying workflow structures on the performance of the algorithms under comparison.

For randomly generated workflows, we change a single parameter from α , β , FR and CCR while keeping the other parameters fixed in each group of experiments. Table III presents the parameters and their values for the simulations.

2) *Performance Impact of Deadlines:* First, we study the performance impact of the deadline factor α , which increases from 0.5 to 2.5 in steps of 0.5. β , FR and CCR are set to 0.3, 7.5% and 1 respectively.

The impact on QTC as a function of the deadline factor α is shown in Fig. 2, which shows that the QTC s of all algorithms show an increasing trend with an increase in α . The reason lies in that the available time slots for each task become looser and looser with the increase in α . As a result, more and more tasks are finished before the deadlines of workflow instances. For each α , Fig. 2 also shows that HFWPF and NEHFWPF yield similar results of QTC s, and both of them are better than the other algorithms. This result is attributed to the fact that both uncertain task execution time and resource failure are considered in HFWPF and NEHFWPF, while only one issue is taken into account in either NOSF or ICFWS. Any unconsidered aspect will inevitably lead to a decline in QTC . The unaccounted fluctuation of task execution time lowers the QTC s of ICFWS, which

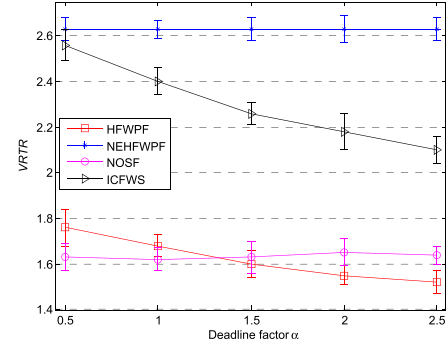
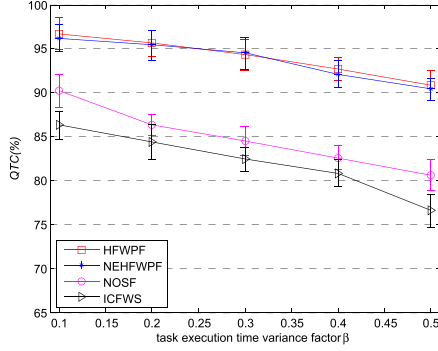
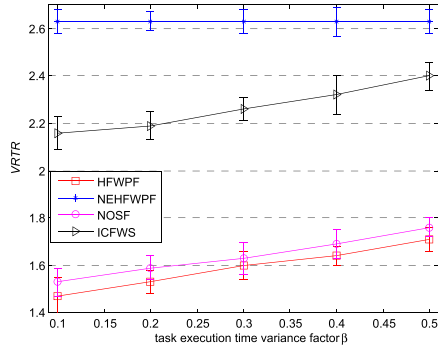


Fig. 3. Performance impact on $VRTR$ as a function of α .

solely considers resource failure. The unconsidered resource failure brings down the QTC s of NOSF, which only focuses on uncertain task execution time. Although NOSF reallocates the task on failed VM to another suitable VM for fault tolerance, the process of finding suitable VM for reallocation takes some time and thus brings down the QTC s of NOSF.

The impact on $VRTR$ as a function of the deadline factor α is shown in Fig. 3, which indicates that both HFWPF and ICFWS have a decreasing trend of $VRTR$ s with an increase in α while the other two algorithms maintain essentially unchanged $VRTR$ values. Fig. 3 also indicates that NEHFWPF has the highest $VRTR$ values under each α and ICFWS gets the intermediate results. Furthermore, NOSF produces the best results of $VRTR$ s for the first two levels of α while HFWPF obtains the best results at the other levels. This phenomenon can be attributed to the different resource provision schemes and fault tolerance strategies employed by each algorithm. NEHFWPF is the only algorithm that keeps all VMs active without elastic resource provisioning, resulting in the highest and most stable $VRTR$ s. ICFWS, HFWPF, and NEHFWPF adopt both replication and resubmission for fault tolerance. As the workflow deadline becomes tighter (smaller α), more tasks choose to use replication for fault tolerance. Conversely, as the workflow deadline becomes looser (larger α), more tasks switch from replication to resubmission, which consumes less resources than replication [4], [8]. Therefore, as α increases, the consumed resources decrease, leading to the decreasing trend in $VRTR$ values observed in HFWPF and ICFWS. However, HFWPF considers both resource failure and uncertain task execution time, while ICFWS only focuses on fault tolerance. The unconsidered uncertain task execution time in ICFWS leads to higher resource consumption and thus higher $VRTR$ values. NOSF is designed for cloud workflow scheduling with uncertain task execution time and resubmits tasks on failed resources in our simulations. NOSF achieves the best $VRTR$ values when α is less than 1.5 because it only uses resubmission for fault tolerance, while the other algorithms use both replication and resubmission when α is less than 1.5. When α is no less than 1.5, almost all tasks use resubmission for faults. In this case, the different VM active schemes result in different $VRTR$ values. NOSF keeps VMs active during their leased time intervals, with some unused time slots within these intervals. In contrast, HFWPF keeps VMs active only when they are executing tasks or waiting for tasks with a waiting time less than T_{boot} . If not, the VMs are shut down

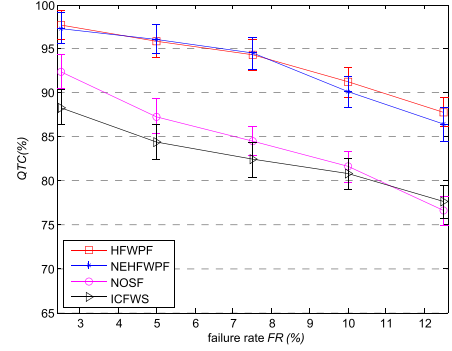
Fig. 4. Performance impact on QTC as a function of β .Fig. 5. Performance impact on $VRTR$ as a function of β .

immediately. Therefore, HFWPF achieves better $VRTR$ values than NOSF when α is no less than 1.5.

By considering both QTC and $VRTR$, it is evident that the proposed HFWPF outperforms all other tested algorithms in terms of higher task completion quality and lower resource consumption. This is beneficial for both users and resource providers in cloud environments as users can receive satisfactory services for their submitted workflows while resource providers can serve more users with the same resources, thereby increasing their revenues [4], [7]. Therefore, HFWPF is more valuable compared to its competitors for both users and resource providers.

3) *Performance Impact of Resource Performance Fluctuation:* We now focus on the performance impact of the task execution time variance factor β , which increases from 0.1 to 0.5 in steps of 0.1. α , FR and CCR are set to 1.5, 7.5% and 1 respectively.

The impact on QTC as a function of β is shown in Fig. 4, which shows that all algorithms keep a decreasing trend of QTC with the increase of β . This is because fluctuations in resource performance cause the resource processing speed to decrease, resulting in lower than normal performance. Consequently, the average task execution time increases with the increase of β , leading to a decline in QTC for all algorithms. Fig. 4 also reveals that HFWPF and its variant NEHFWPF have similar QTC values and both of them outperform NOSF and ICFWS, which have the lowest QTC values among all algorithms. The reasons for this are the same as those explained in Fig. 2.

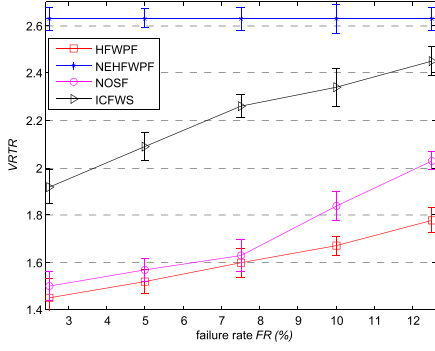
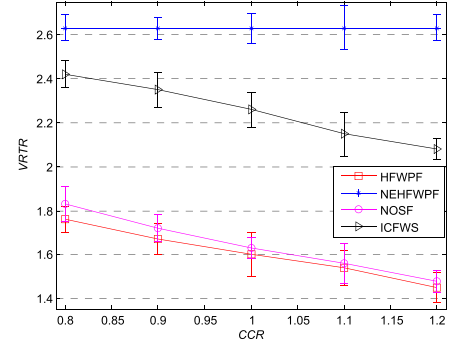
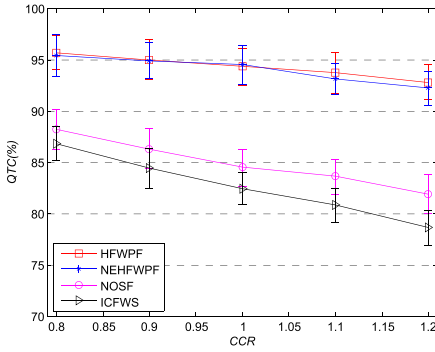
Fig. 6. The impact on QTC as a function of FR .

The impact on $VRTR$ as a function of β is shown in Fig. 5, which indicates that NEHFWPF maintains constant $VRTR$ values with the increase of β while the others exhibit an increasing trend in $VRTR$. Furthermore, Fig. 5 also indicates that HFWPF achieves the best $VRTR$ results among all algorithms. NEHFWPF keeps all VMs active regardless of resource performance, resulting in the highest and consistent $VRTR$ values. For the other three algorithms, fluctuations in resource performance bring down the resource processing speed, as explained in Fig. 4. This leads to increased resource consumption and a subsequent decrease in their $VRTR$ values. The reasons for HFWPF's superior $VRTR$ results are the same as those explained in Fig. 3.

4) *Performance Impact of Resource Failures:* We now turn to the performance impact of the VM failure rate FR , which increases from 2.5% to 12.5% in steps of 2.5%. α , β and CCR are set to 1.5, 0.3 and 1 respectively.

The impact of failure rates FR on QTC is illustrated in Fig. 6, which shows that all algorithms exhibit a decreasing trend in QTC as FR increases. This is because all algorithms employ resubmission as a fault-tolerant strategy for either all or partial tasks. Specifically, HFWPF, NEHFWPF, and ICFWS use resubmission for partial tasks, while NOSF employs it for all tasks. As FR increases, more resources fail, necessitating more resubmissions. Consequently, fewer tasks are completed by their sub-deadlines, leading to lower QTC values. Fig. 6 also reveals that HFWPF and NEHFWPF achieve similar QTC values and outperform NOSF and ICFWS. This is because HFWPF and NEHFWPF consider both resource failure and performance fluctuation, whereas NOSF and ICFWS only consider one of these factors.

The impact of FR on $VRTR$ is shown in Fig. 7, which indicates that NEHFWPF keeps stable and the highest $VRTR$ values under different FR s while HFWPF, NOSF and ICFWS have an increase tendencies in $VRTR$ with the increase in FR . The NEHFWPF holds stable and the highest $VRTR$ values. The reason lies in that all VMs in NEHFWPF are kept active, no matter how many failures happen. The other three algorithms adopt elastic resource provisioning mechanism and active VMs according to the scheduling requirement. Moreover, they all take resubmission as a fault-tolerant strategy for either all or partial tasks. As FR increases, more and more tasks, which select resubmission as fault-tolerant strategy in initial scheduling phase, fail and

Fig. 7. The impact on $VRTR$ as a function of FR .Fig. 9. The impact on $VRTR$ as a function of CCR .Fig. 8. The impact on QTC as a function of CCR .

then require additional VMs for resubmission. Consequently, more VMs are activated, leading to larger $VRTR$ values. Fig. 7 also reveals that HFWPF has the best $VRTR$ values among all algorithms. The reasons for this are the same as those explained in Fig. 3.

5) *Performance Impact of CCRs*: We also study the performance impact of CCR , which increases from 0.8 to 1.2 in steps of 0.1. α , β and FR are set to 1.5, 0.3 and 7.5% respectively.

The influence of CCR on QTC is depicted in Fig. 8, from which we can observe that the proposed HFWPF achieves the best results among all compared algorithms, despite the fact that all algorithms exhibit large QTC values for different CCR s. The reasons are similar to those explained for Fig. 3. Fig. 8 also indicates that each of the compared algorithms exhibits a decreasing trend in QTC with an increase in CCR . The reason for this lies in the fact that the number of edges, along with the dependencies among tasks, increases with the increase of CCR . The increasing interdependencies between tasks can lead to a situation where the delayed completion time of one task, caused by resource failures or uncertain task execution durations, affects an ever-growing number of other tasks. Therefore, the QTC s decrease with an increase in CCR .

The influence of CCR on $VRTR$ is depicted in Fig. 9, illustrating that all compared algorithms exhibit a decreasing trend in $VRTR$ as CCR increases. The reason for this is that a VM allocated to a task must remain active for either task communication or task execution. An increase in CCR corresponds to an increase in average communication time and a decrease in average task execution time. The reduction in task execution time results in

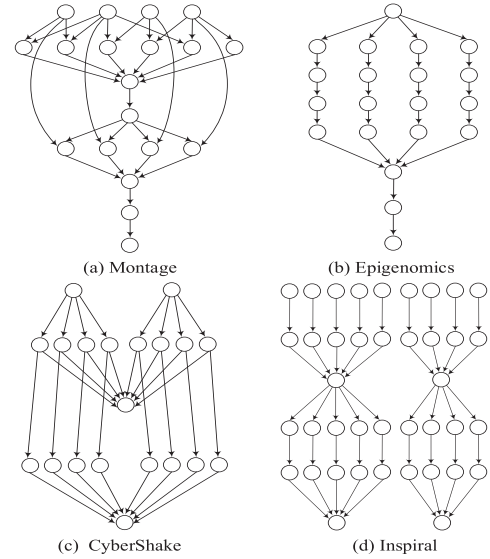


Fig. 10. The structures of four real-world workflows.

smaller $VRTR$ values for all compared algorithms. Additionally, the average communication time further diminishes $VRTR$ s, as successful communication requires at least two active resources, leading to the use of more VMs. Nevertheless, HFWPF achieves the best $VRTR$ s among all compared algorithms across different CCR s, confirming that HFWPF consumes the fewest resources.

B. Evaluation on Simulated Cloud Platform With Real-World Workflows

Besides randomly generated workflows, four real-world workflows as shown in Fig. 10, i.e., Montage, CyberShake, Epigenomics and Inspiral [41], are selected to evaluate the algorithms. These workflows have been widely used to measure the performance of scheduling algorithms [1], [2], [3], [4]. The features of these instances used in our experiments are given in Table IV. More detail on these workflows were given in [41]. The deadline of each workflow instance, the expected value and variance of task execution time are set by Exp. (14) and Exp. (15), where α and β are set to 1.5 and 0.3 respectively. FR is set to 7.5% in this section.

TABLE IV
PARAMETERS FOR THE REAL-WORLD WORKFLOWS

Workflow	Number of tasks	Number of edges	Average data size (MB)	Average task runtime (s)
Montage	1000	4485	3.21	11.36
Epigenomics	997	3228	388.59	3858.67
CyberShake	1000	3988	102.29	22.71
Inspirial	1000	3246	8.90	227.25

* When calculating the number of edges, average data size and average task runtime, the source/sink node and the related edges are included.

TABLE V
PERFORMANCE FOR REAL-WORLD WORKFLOWS

Workflow	Metric	HFWPF	NEHFWPF	NOSF	ICFWS
Montage	<i>QTC</i> (%)	93.9	94.2	85.3	82.8
	<i>VRTR</i>	1.65	2.74	1.72	2.3
Epigenomics	<i>QTC</i> (%)	93.5	93.3	84.5	83.1
	<i>VRTR</i>	1.63	2.65	1.70	2.1
CyberShake	<i>QTC</i> (%)	90.5	90.6	81.3	79.7
	<i>VRTR</i>	1.43	2.58	1.51	1.88
Inspirial	<i>QTC</i> (%)	93.8	93.6	81.3	79.7
	<i>VRTR</i>	1.63	2.87	1.66	2.02

The experimental results are presented in Table V, which demonstrate that HFWPF outperforms other algorithms in practical scenarios. In terms of *QTC*s, HFWPF achieves the best outcomes for two real-world workflows: Epigenomics and Inspirial, while NEHFWPF yields the best results for Montage and CyberShake. Furthermore, both HFWPF and NEHFWPF surpass the other two algorithms. NEHFWPF obtains the highest *QTC* values on Montage and CyberShake because all available VMs remain active, and no initial time is required to boot them. However, as shown in Table V, HFWPF is only slightly lower than NEHFWPF on Montage and CyberShake and performs better on the other workflows. This can be attributed to the effective algorithm used in HFWPF for handling resource failures and performance fluctuations, as discussed in Section IV. Regarding *VRTR*s, HFWPF also achieves the best results. This observation once again highlights that HFWPF can effectively reduce resource consumption due to its elastic resource provisioning and VM shutdown mechanisms. The results further indicate that HFWPF can enhance resource utilization and decrease resource costs by fully utilizing existing resources while accounting for both resource failures and performance fluctuations.

C. Evaluation on Real Cloud Platform

We also conduct experiments on a real cloud platform to evaluate the performance of the compared algorithms.

The cloud platform under consideration comprises 18 diverse hosts, inclusive of 13 Sugon I620-G10, 1 Sugon I620-G15, 2 Dell R720, and 2 Dell T630. The detailed specifications of these hosts are meticulously enumerated in Table VI. All hosts are seamlessly connected via an H3C S5100 switch, ensuring efficient network communication. Each host is virtualized to multiple VMs. Four types of VM are created in this experiment. The specifications for each type of VM are methodically

TABLE VI
THE CONFIGURATION OF HOSTS

Server model	Number of servers	CPU	Memory (DDR3)	Storage capacity
Sugon I620-G10	13	2 × Intel Xeon E5-2630 2.3GHz	64G	1.08TB
Sugon I620-G15	1	2 × Intel Xeon E5-2609 2.4GHz	128G	2.94TB
Dell R720	2	2 × Intel Xeon E5-2640 2.0GHz	128G	3.81TB
Dell T630	2	2 × Intel Xeon E5-2650 2.2GHz	64G	4.47TB

TABLE VII
THE CONFIGURATION OF VMs

Type	vCPU	Memory	Storage
Type 1	1	2G	16G
Type 2	2	4G	32G
Type 3	4	8G	64G
Type 4	8	16G	128G

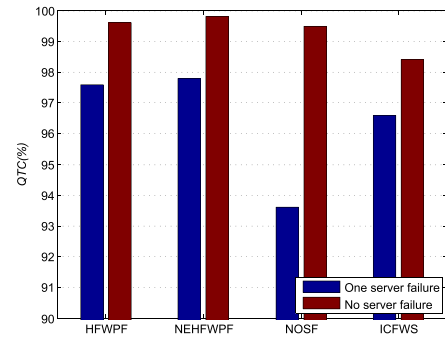


Fig. 11. The *QTC* results on real cloud platform.

outlined in Table VII. We consider 160,000 face recognition collected by a community access control system in 10 days as experimental data. Each face recognition includes multiple steps such as image acquisition, face detection, pre-processing, feature extraction, feature matching, recognition decision and log update. Therefore, they are workflow instances. The deadline for each recognition is set to 2 seconds. During the experiment, we unplugged the network cable of one Sugon I620-G10 server to simulate resource failure. To clearly observe the overhead caused by resource failures, we also conducted corresponding experiments under condition with no resource failures.

The results about *QTC* on real cloud platform is shown in Fig. 11, which indicates that all compared algorithms exhibited a significant decrease in *QTC* values after the occurrence of resource failures. Fig. 11 also shows that ICFWS achieves the worst results if no server fails, while the other three algorithms yield similar outcomes and NEHFWPF achieves the best results. HFWPF and NEHFWPF yield similar *QTC* results, and both outperform the other comparative algorithms when one server fails. NOSF yields the worst results among all compared algorithms when one server fails. The reason lies in the fact that HFWPF and its variant, NEHFWPF, are tailored for cloud workflow

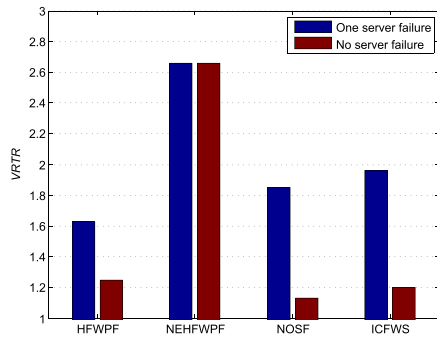


Fig. 12. The *VRTR* results on real cloud platform.

scheduling with both resource failures and uncertain task execution time. ICFWS and NOSF are designed for cloud workflow scheduling with resource failures and uncertain task execution time, respectively. However, ICFWS does not consider uncertain task execution time, while NOSF does not consider resource failures. When comparing HFWPF and NEHFWPF, regardless of resource failures, NEHFWPF consistently achieves better *QTC* results than HFWPF. This is because all VMs in NEHFWPF keep in an active state, reducing the setup times of VMs and consequently improving their *QTC* values.

The results regarding *VRTR* on a real cloud platform are illustrated in Fig. 12, which shows that HFWPF achieves the best *VRTR* results among all algorithms in the presence of resource failures, while NEHFWPF obtains the worst *VRTR* results. When no resource failures occurred, NOSF achieves the best *VRTR* results, whereas NEHFWPF still obtains the worst *VRTR* results, maintaining the same *VRTR* performance as in the case of resource failures. This is because both HFWPF and ICFWS adopt elastic resource provisioning and hybrid fault-tolerance mechanisms. However, ICFWS only considers resource failures without accounting for uncertain task execution time. Although NEHFWPF also employs a hybrid fault-tolerance strategy, its VMs remain perpetually active. When no resource failures occurred, NOSF does not reserve any resources for failures, while the other three algorithms reserve resources for task replicas during scheduling as replication is adopted in these algorithms, thereby improving their *VRTR* values.

Upon simultaneously observing Figs. 11 and 12, it becomes evident that HFWPF outperforms both NOSF and ICFWS in terms of both *QTC* and *VRTR*. When comparing HFWPF with NEHFWPF, it is noted that NEHFWPF achieves a slight lead over HFWPF in *QTC*, but at the expense of a significant *VRTR* penalty. Consequently, HFWPF exhibits superior comprehensive performance over NEHFWPF in actual cloud platforms.

In this study, the VM fault detection time is ignored. This assumption is grounded in the efficiency of modern cloud monitoring systems (e.g., those based on heartbeats or fail-signals), which can typically detect a VM failure within a very short time interval, often on the order of seconds [38], [39]. Given that the VM boot time ($T_{boot} = 15s$) and the task execution times (typically in the order of minutes) are the dominant factors in the scheduling timeline, the inclusion of a second detection latency would have a negligible impact on the overall comparative

results of the algorithms, which is the primary focus of this performance evaluation. However, we acknowledge that this simplification might lead to a slight underestimation of the task resubmission overhead in real-world systems. The performance gap introduced by this assumption is expected to be consistent across all compared algorithms, as they all would be affected by a similar fixed detection delay upon a fault. Therefore, while the absolute values of metrics like *QTC* and *VRTR* might be marginally more optimistic, the relative performance trends and the superiority of HFWPF demonstrated in our experiments are expected to hold robustly.

VI. CONCLUSIONS AND FUTURE RESEARCH

To deal with fault-tolerant cloud workflow scheduling with performance fluctuated resources, we have proposed a workflow scheduling algorithm called HFWPF in this paper. It consists of four phases, including preprocessing, initial scheduling, online scheduling and online adjustment. The preprocessing divides the deadline of one accepted workflow instance into multiple subdeadlines for all tasks. The initial scheduling selects a fault-tolerant strategy from either replication or resubmission for each task. It also selects served VM for each task. For tasks with resubmission, the online scheduling is used for online resubmission for faults. The online adjustment resizes the subdeadlines of some unexecuted tasks to make full use of time slots. Furthermore, HFWPF also adopts an elastic resource provisioning mechanism to satisfy the dynamic scheduling requirements. The experiments show that HFWPF can enhance the resource utilization and guarantee workflow execution time simultaneously in the context of fault tolerance and performance fluctuation.

The choice of fault-tolerant strategy from resubmission and replication for each task is only decided by the task slack time in this work. Expanding this research to include some other factors, such as cost or reliability, is a promising avenue for future work. Furthermore, VM migration and consolidation is an effective technique for reducing energy consumption and enhancing resource utilization. Extending this work to consider VM migration and consolidation is a promising avenue for future work.

REFERENCES

- [1] H. Chen, X. Zhu, G. Liu, and W. Pedrycz, "Uncertainty-aware online scheduling for real-time workflows in cloud service environment," *IEEE Trans. Serv. Comput.*, vol. 14, no. 4, pp. 1167–1178, Jul./Aug. 2021.
- [2] J. Liu et al., "Online multi-workflow scheduling under uncertain task execution time in IaaS clouds," *IEEE Trans. Cloud Comput.*, vol. 9, no. 3, pp. 1180–1194, Third Quarter 2021.
- [3] Y. Ding, G. Yao, and K. Hao, "Fault-tolerant elastic scheduling algorithm for workflow in cloud systems," *Inf. Sci.*, vol. 393, pp. 47–65, 2017.
- [4] G. Yao, Y. Ding, and K. Hao, "Using imbalance characteristic for fault-tolerant workflow scheduling in cloud systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 12, pp. 3671–3683, Dec. 2017.
- [5] H. Chen, J. Zhu, Z. Zhang, M. Ma, and X. Shen, "Real-time workflows oriented online scheduling in uncertain cloud environment," *J. Supercomputing*, vol. 73, no. 11, pp. 4906–4922, 2017.
- [6] J. Schad, J. Dittrich, and J.-A. Quiané-Ruiz, "Runtime measurements in the cloud: Observing, analyzing, and reducing variance," *Proc. VLDB Endowment*, vol. 3, no. 1/2, pp. 460–471, 2010.

- [7] G. Yao, Q. Ren, X. Li, S. Zhao, and R. Ruiz, "A hybrid fault-tolerant scheduling for deadline-constrained tasks in cloud system," *IEEE Trans. Serv. Comput.*, vol. 15, no. 3, pp. 1371–1384, May/Jun. 2022.
- [8] K. Plankensteiner and R. Prodan, "Meeting soft deadlines in scientific workflows using resubmission impact," *IEEE Trans. Parallel Distrib. Syst.*, vol. 23, no. 5, pp. 890–901, May 2012.
- [9] N. Yigitbasi, O. Sonmez, A. Iosup, and D. Epema, "Performance evaluation of overload control in multi-cluster grids," in *Proc. IEEE/ACM 12th Int. Conf. Grid Comput.*, 2011, pp. 173–180.
- [10] D. Cotroneo, R. Natella, and S. Rosiello, "Overload control for virtual network functions under CPU contention," *Future Gener. Comput. Syst.*, vol. 99, pp. 164–176, 2019.
- [11] W. Qiu, Z. Zheng, X. Wang, X. Yang, and M. R. Lyu, "Reliability-based design optimization for cloud migration," *IEEE Trans. Serv. Comput.*, vol. 7, no. 2, pp. 223–236, Second Quarter 2014.
- [12] B. Javadi, D. Kondo, J. Vincent, and D. P. Anderson, "Discovering statistical models of availability in large distributed systems: An empirical study of SETI@home," *IEEE Trans. Parallel Distrib. Syst.*, vol. 22, no. 11, pp. 1896–1903, Nov. 2011.
- [13] R. Jhawar, V. Piuri, and M. D. Santambrogio, "Fault tolerance management in cloud computing: A system-level perspective," *IEEE Syst. J.*, vol. 7, no. 2, pp. 288–297, Jun. 2013.
- [14] B. Wu, K. Hao, X. Cai, and T. Wang, "An integrated algorithm for multi-agent fault-tolerant scheduling based on MOEA," *Future Gener. Comput. Syst.*, vol. 94, pp. 51–61, 2019.
- [15] G. Yao, Y. Ding, L. Ren, K. Hao, and L. Chen, "An immune system-inspired rescheduling algorithm for workflow in cloud systems," *Knowl. Based Syst.*, vol. 99, no. 99, pp. 39–50, 2016.
- [16] X. Zhu, J. Wang, H. Guo, D. Zhu, L. T. Yang, and L. Liu, "Fault-tolerant scheduling for real-time scientific workflows with elastic resource provisioning in virtualized clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 12, pp. 3501–3517, Dec. 2016.
- [17] Z. Li, H. Yu, G. Fan, and J. Zhang, "Cost-efficient fault-tolerant workflow scheduling for deadline-constrained microservice-based applications in clouds," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 3220–3232, Sep. 2023.
- [18] T. Dong, F. Xue, H. Tang, and C. Xiao, "Deep reinforcement learning for fault-tolerant workflow scheduling in cloud environment," *Appl. Intell.*, vol. 53, no. 9, pp. 9916–9932, 2023.
- [19] M. Farid, R. Latip, M. Hussin, and N. A. W. A. Hamid, "A fault-intrusion-tolerant system and deadline-aware algorithm for scheduling scientific workflow in the cloud," *PeerJ Comput. Sci.*, vol. 7, 2021, Art. no. e747.
- [20] A. R. Setlur, S. J. Nirmala, H. S. Singh, and S. Khoriya, "An efficient fault tolerant workflow scheduling approach using replication heuristics and checkpointing in the cloud," *J. Parallel Distrib. Comput.*, vol. 136, pp. 14–28, 2020.
- [21] Z. Li, V. I. Chang, H. Hu, H. Hu, C. Li, and J. Ge, "Real-time and dynamic fault-tolerant scheduling for scientific workflows in clouds," *Inf. Sci.*, vol. 568, pp. 13–39, 2021.
- [22] Z. Cai, X. Li, R. Ruiz, and Q. Li, "A delay-based dynamic scheduling algorithm for bag-of-task workflows with stochastic task execution times in clouds," *Future Gener. Comput. Syst.*, vol. 71, pp. 57–72, 2017.
- [23] M. A. Rodriguez and R. Buyya, "Deadline based resource provisioning and scheduling algorithm for scientific workflows on clouds," *IEEE Trans. Cloud Comput.*, vol. 2, no. 2, pp. 222–235, Second Quarter 2014.
- [24] R. N. Calheiros and R. Buyya, "Meeting deadlines of scientific workflows in public clouds with tasks replication," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 7, pp. 1787–1796, Jul. 2014.
- [25] L. Xing, M. Zhang, H. Li, M. Gong, J. Yang, and K. Wang, "Local search driven periodic scheduling for workflows with random task runtime in clouds," *Comput. Ind. Eng.*, vol. 168, 2022, Art. no. 108033.
- [26] J. Xu, H. Yu, G. Fan, and J. Zhang, "Uncertainty-aware scheduling of real-time workflows under deadline constraints on multi-cloud systems," *Concurrency Computation: Pract. Experience*, vol. 35, no. 5, 2023, Art. no. e7562.
- [27] K. Y. Rajput, X. Li, J. Zhang, and A. Lakhan, "A novel scheduling approach for spark workflow tasks with deadline and uncertain performance in multi-cloud networks," *IEEE Trans. Cloud Comput.*, vol. 12, no. 4, pp. 1145–1157, Fourth Quarter 2024.
- [28] X. Chen, C. Lin, and B. Lin, "An intelligent workflow scheduling scheme for complex network robustness in fuzzy edge-cloud environments," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 1, pp. 1106–1123, Jan./Feb. 2024.
- [29] B. Lin, C. Lin, X. Chen, M. Lin, G. Huang, and Z. Xu, "Cost-driven scheduling for workflow decision making systems in fuzzy edge-cloud environments," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 3756–3771, 2025.
- [30] Y. Pan et al., "A novel approach to scheduling workflows upon cloud resources with fluctuating performance," *Mobile Netw. Appl.*, vol. 25, pp. 690–700, 2020.
- [31] J. Bader, F. Lehmann, L. Thamsen, U. Leser, and O. Kao, "Lotaru: Locally predicting workflow task runtimes for resource management on heterogeneous infrastructures," *Future Gener. Comput. Syst.*, vol. 150, pp. 171–185, 2024.
- [32] Y. Yang, H. Shen, and H. Tian, "Scheduling workflow tasks with unknown task execution time by combining machine-learning and greedy-optimization," *IEEE Trans. Serv. Comput.*, vol. 17, no. 3, pp. 1181–1195, May/Jun. 2024.
- [33] S. Qin, D. Pi, Z. Shao, and Y. Xu, "A discrete interval-based multi-objective memetic algorithm for scheduling workflow with uncertainty in cloud environment," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 3020–3037, Sep. 2023.
- [34] L. Ye, Y. Xia, L. Yang, and C. Yan, "SHWS: Stochastic hybrid workflows dynamic scheduling in cloud container services," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 3, pp. 2620–2636, Jul. 2022.
- [35] A. Taghinezhad-Niar, S. Pashazadeh, and J. Taheri, "QoS-aware online scheduling of multiple workflows under task execution time uncertainty in clouds," *Cluster Comput.*, vol. 25, no. 6, pp. 3767–3784, 2022.
- [36] S. Abrishami, M. Naghibzadeh, and D. H. Epema, "Cost-driven scheduling of grid workflows using partial critical paths," *IEEE Trans. Parallel Distrib. Syst.*, vol. 23, no. 8, pp. 1400–1414, Aug. 2012.
- [37] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.
- [38] D.-M. Bui, T. Huynh-The, and S. Lee, "Early fault detection in IaaS cloud computing based on fuzzy logic and prediction technique," *J. Supercomputing*, vol. 74, pp. 5730–5745, 2018.
- [39] Y.-L. Lee, D. Liang, and W.-J. Wang, "Optimal online liveness fault detection for multilayer cloud computing systems," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3464–3477, Sep./Oct. 2022.
- [40] W. Chen and E. Deelman, "WorkflowSim: A toolkit for simulating scientific workflows in distributed environments," in *Proc. IEEE 8th Int. Conf. E-Sci.*, 2012, pp. 1–8.
- [41] E. Deelman et al., "Pegasus, a workflow management system for science automation," *Future Gener. Comput. Syst.*, vol. 46, pp. 17–35, 2015.



Qian Ren received the MS degree from the Hefei University of Technology, Hefei, China, in 2012. She is currently an associate professor with the College of Computer and Information Engineering, Chuzhou University, China. She is the author or co-author more than 10 academic papers, some of which have been published in international journals such as *IEEE Transactions on Services Computing*; *IEEE ACCESS*; *Sensors*. Her research interests include cloud resource scheduling and Internet of Things.



Guangshun Yao received the PhD degree from Donghua University, China, in 2016. He is a professor with the College of Computer and Information Engineering, Chuzhou University, China. He is the author or co-author more than 50 academic papers, some of which have been published in international journals such as *IEEE Transactions on Parallel and Distributed Systems*; *IEEE Transactions on Services Computing*; *Information Sciences*; *Knowledge-Based Systems*; *Soft Computing*. His research interests include cloud computing and Internet of Things.